# Grid Files

(Secondary Key Retrieval)

# Motivation

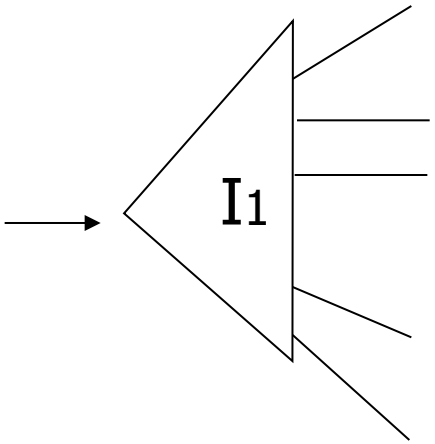
- The grid file is another data structure for providing efficient retrieval of records with multiple attributes.
- Like k-d tree it limits the search space that must be considered for retrieval.

Example: Find records where

DEPT = “Toy” AND SAL > 50k

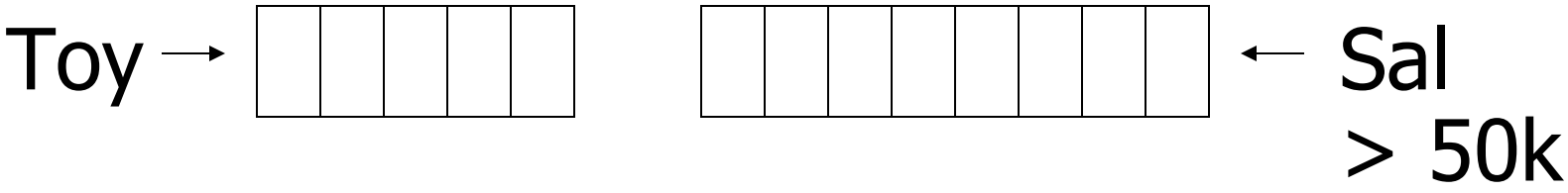
# Strategy I:

- Use one index, say Dept.
- Get all Dept = "Toy" records
  and check their salary



# Strategy II:

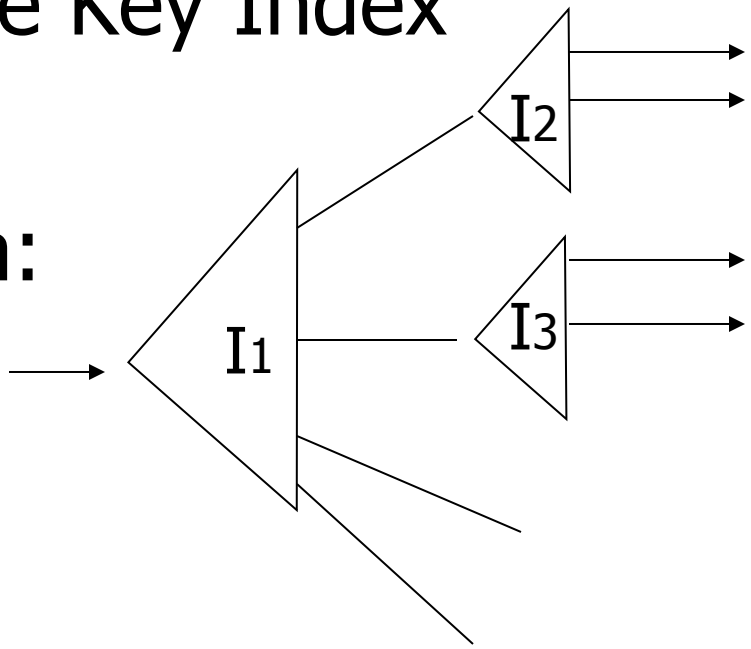
- Use 2 Indexes; Manipulate Pointers



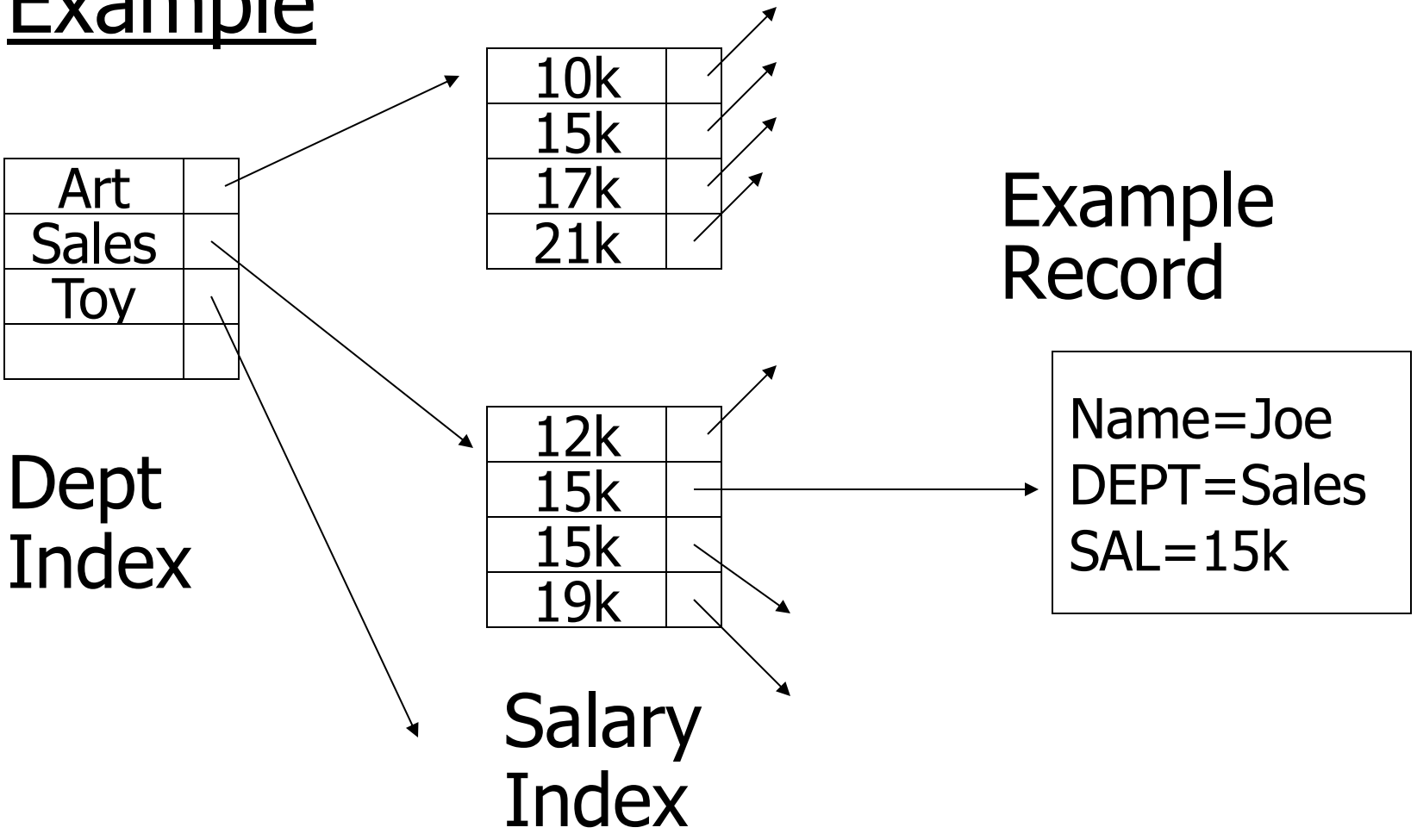
# Strategy III:

- Multiple Key Index

One idea:



# Example

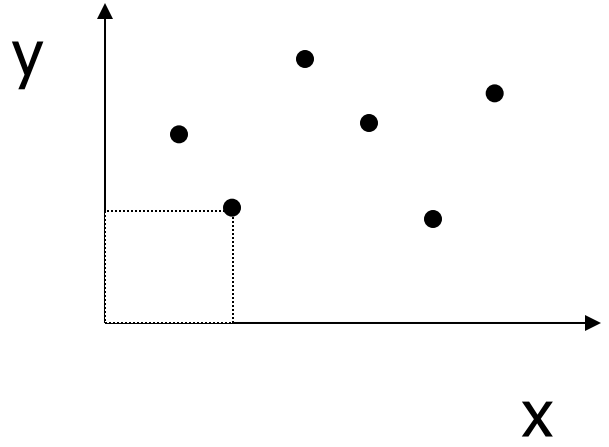

Art
Sales
Toy
10k
15k
17k
21k
12k
15k
15k
19k
Dept Index
Salary Index
Example Record
Name=Joe
DEPT=Sales
SAL=15k

# For which queries is this index good?

- ☐ Find RECs Dept = "Sales" $\wedge$ SAL=20k
- ☐ Find RECs Dept = "Sales" $\wedge$ SAL $\geq$ 20k
- ☐ Find RECs Dept = "Sales"
- ☐ Find RECs SAL = 20k

# Interesting application:

- Geographic Data




DATA:

<$X_1$,$Y_1$, Attributes>

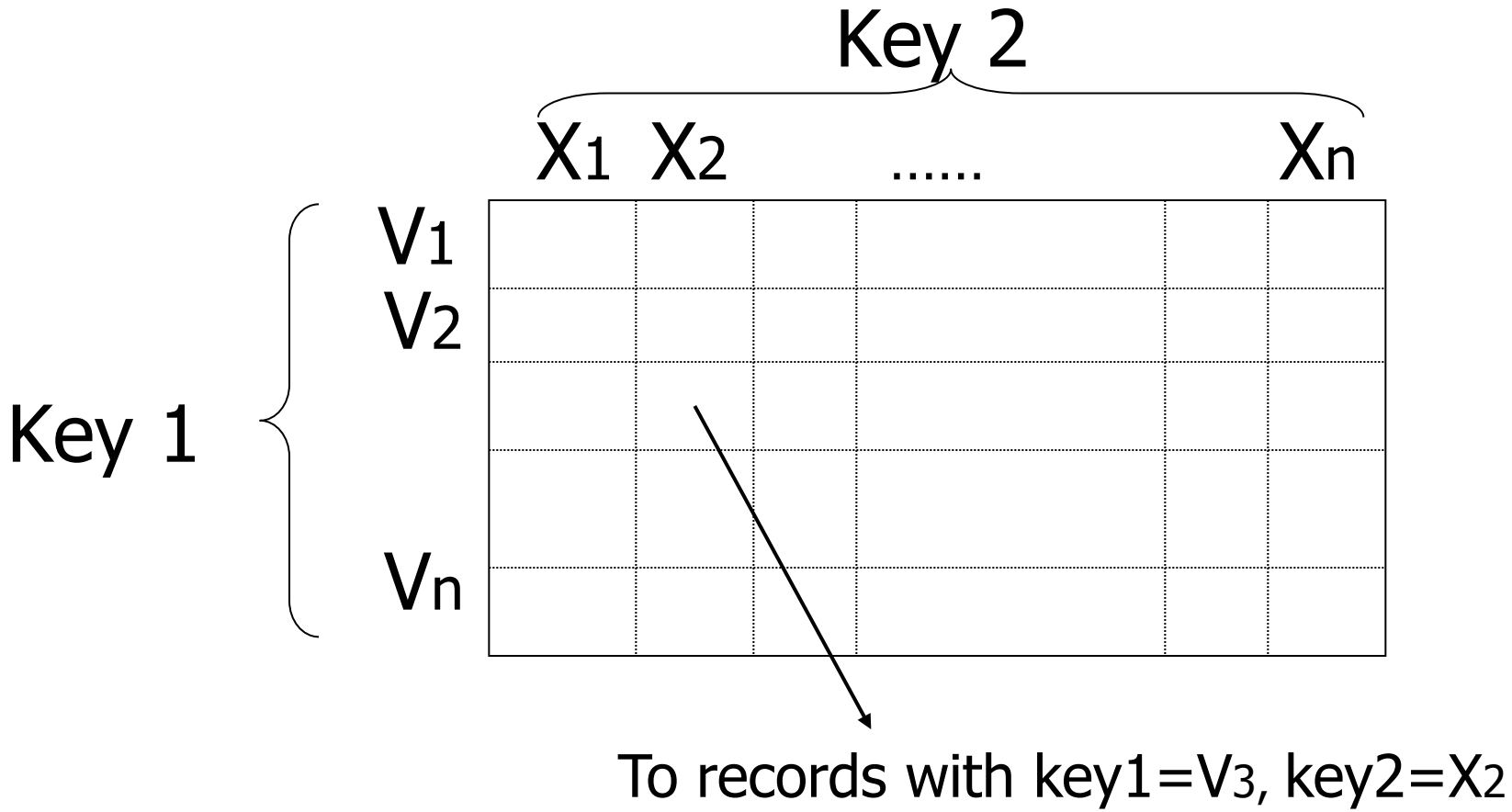
<$X_2$,$Y_2$, Attributes>

⋮

# Queries:

- What city is at <Xi,Yi>?
- What is within 5 miles from <Xi,Yi>?
- Which is closest point to <Xi,Yi>?

# Grid File



# CLAIM

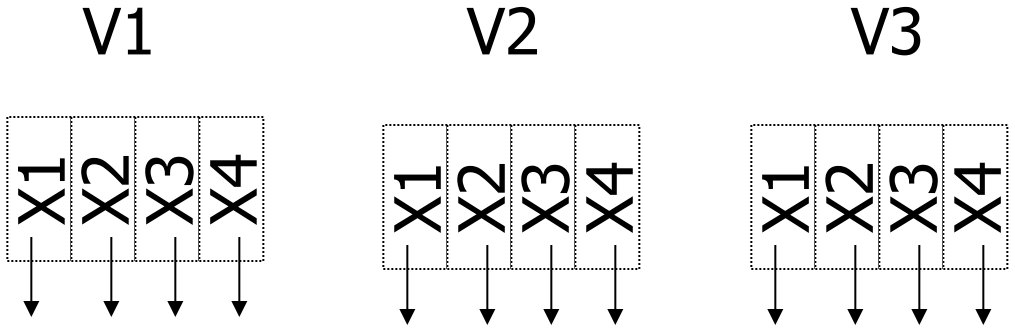
- Can quickly find records with
  - key 1 = $V_i$ $\wedge$ Key 2 = $X_j$
  - key 1 = $V_i$
  - key 2 = $X_j$

- And also ranges….
  - E.g., key 1 $\geq V_i$ $\wedge$ key 2 < $X_j$

✉ But there is a <u>catch</u> with Grid File!

- How is Grid File stored on disk?

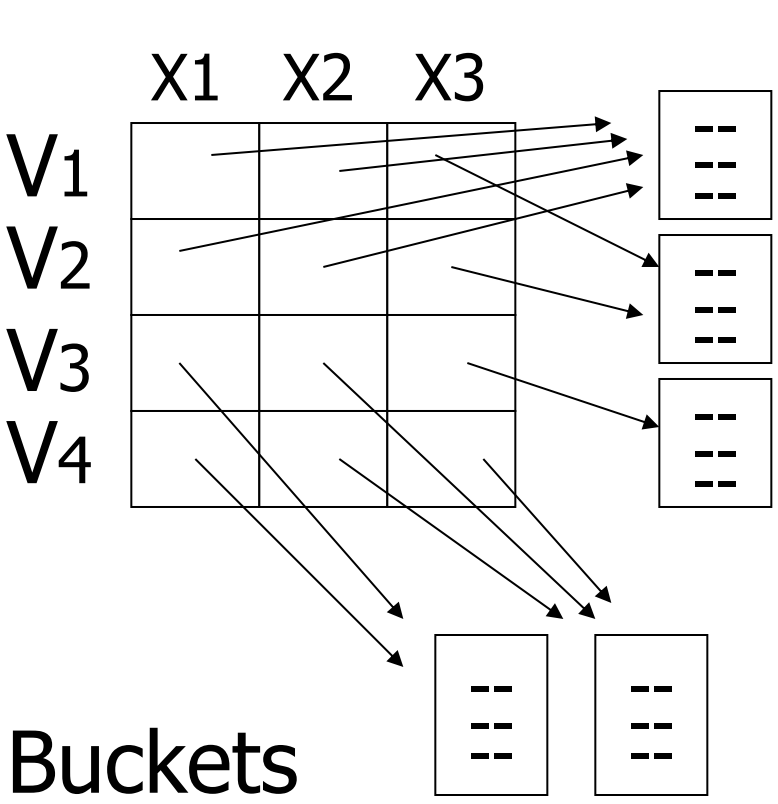
Like Array...




<u>Problem:</u>

- Need regularity so we can compute position of <Vi,Xj> entry

# Solution: Use Indirection

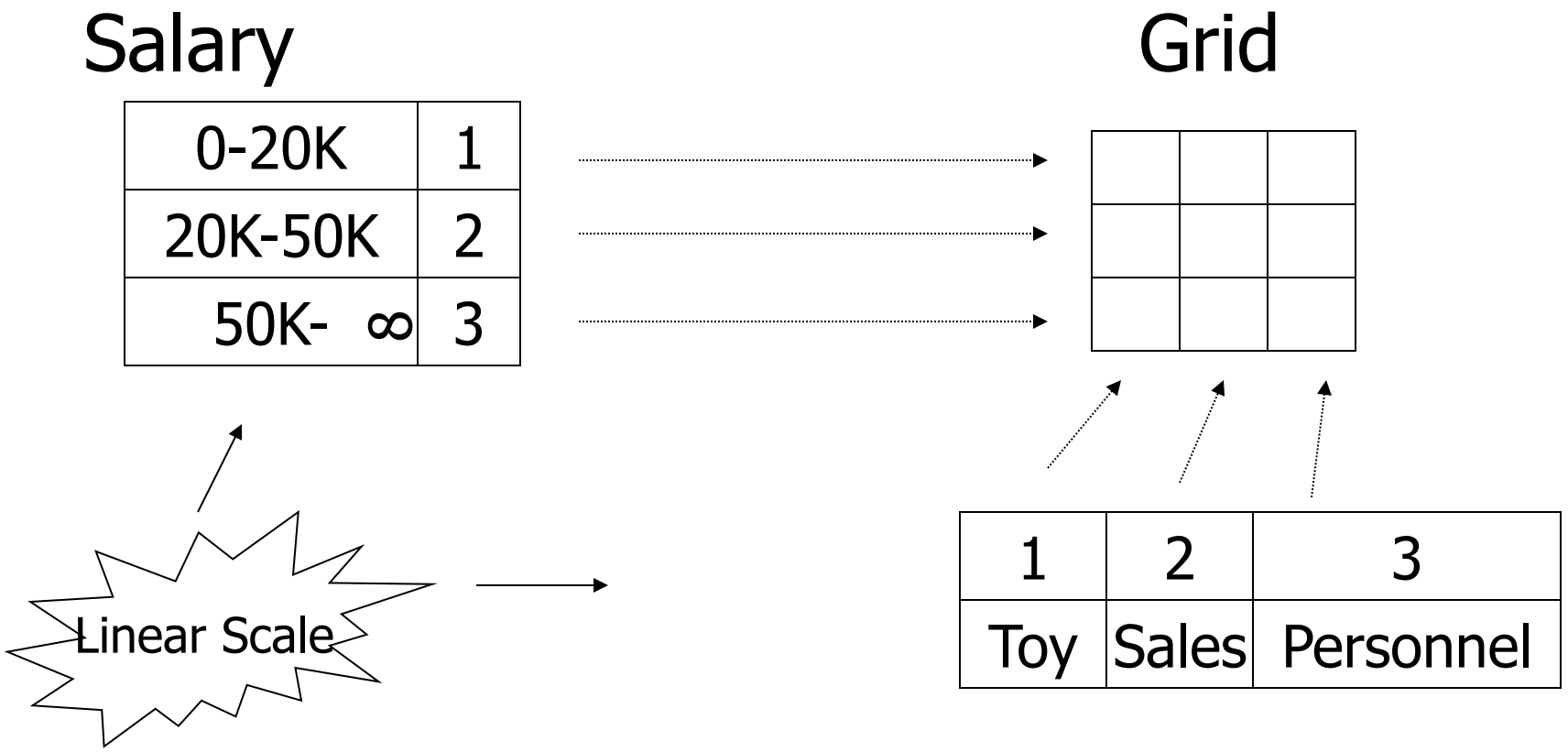



*Grid only contains pointers to buckets

# Grid File

- A grid file organizes data into buckets (pages) corresponding to the intersection points of a multidimensional box.
- An index provides access to the individual buckets.
- If we use 2 attributes for searching, index part is a rectangle to show the 2 dimensional search space.
- If we use d attributes for searching, index is a d dimensional shape.

# Can also index grid on value ranges



# Grid Files

- For each attribute, a linear scale is used for converting the attribute value to the correct element in the grid array.
- Each element in the linear scale represents the maximum value for the interval.
- Linear scales are kept in memory.
- An element in the grid array stores address of the corresponding data bucket.
- If size of the grid array is large, it is stored on disk, otherwise it is kept in memory.
- Buckets are always kept on disk.

# Example

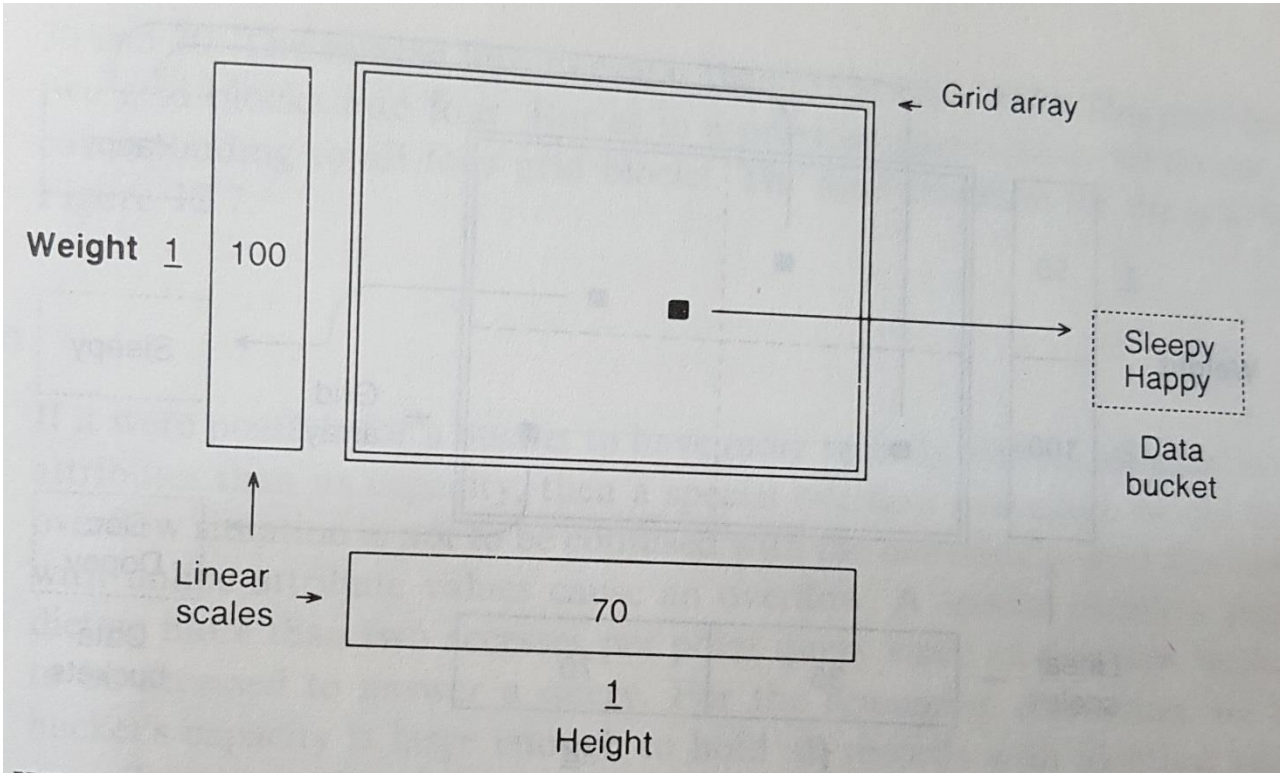
- Representative data records (Ms. White and 7 dwarfs)

| Name | Height | Weight | Other data |
|---|---|---|---|
| Sleepy | 36 | 48 | |
| Happy | 34 | 52 | |
| Doc | 38 | 51 | |
| Dopey | 37 | 54 | |
| Grumpy | 32 | 55 | |
| Sneezy | 35 | 46 | |
| Bashful | 33 | 50 | |
| Ms. White | 62 | 98 | |

# Insertion

- We insert each data record in the given order for this dataset.
- We use 2 attributes (i.e., height and weight), so the grid will be a rectangle (2-d array), we will have 2 linear scales, and 1 data bucket.
- In this example we assume that each data bucket can hold at most 2 data records.
- First insert Sleepy(36, 48), and then Happy(34, 52) as each bucket can store up to 2 records.

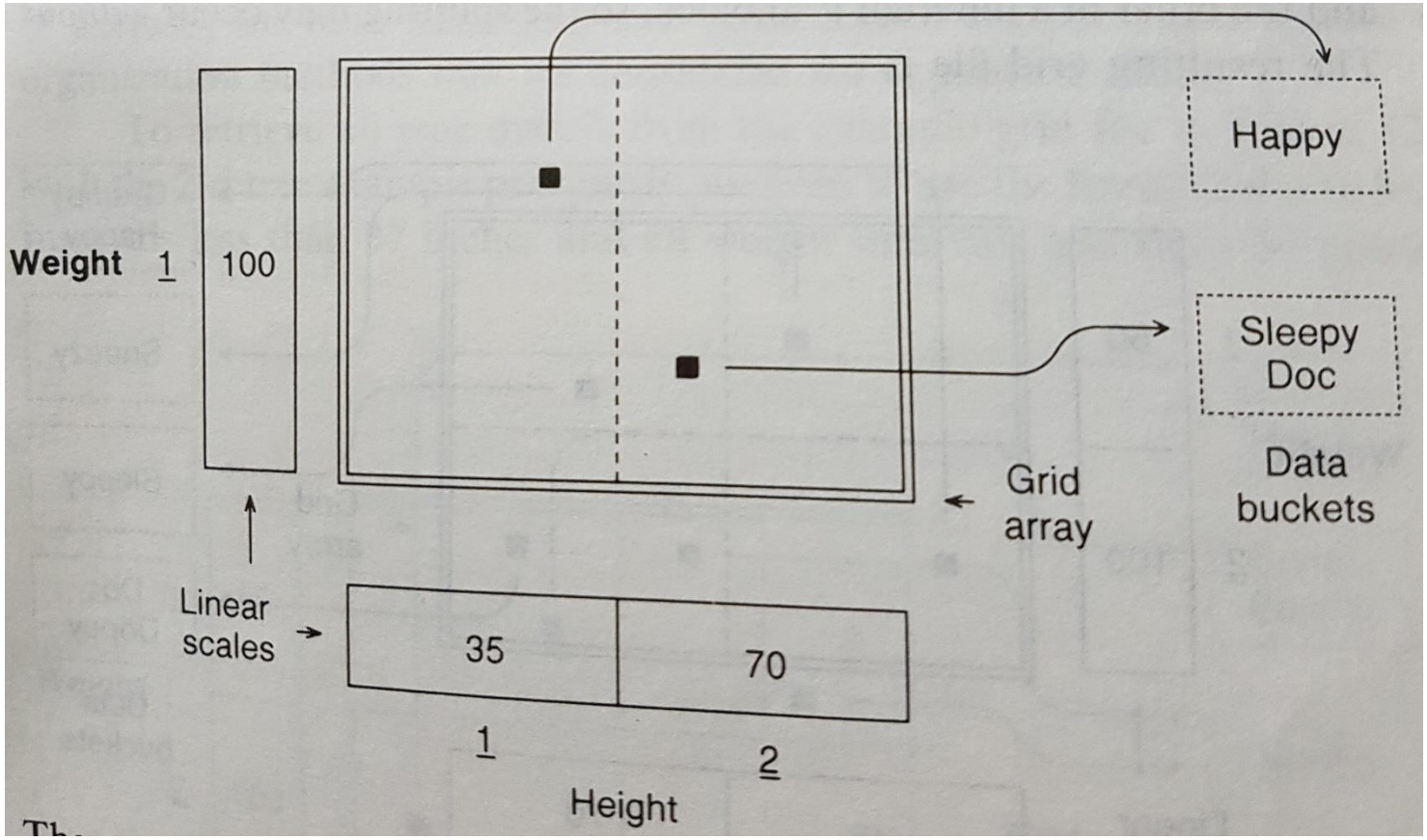
# Insertion (Cont'd)



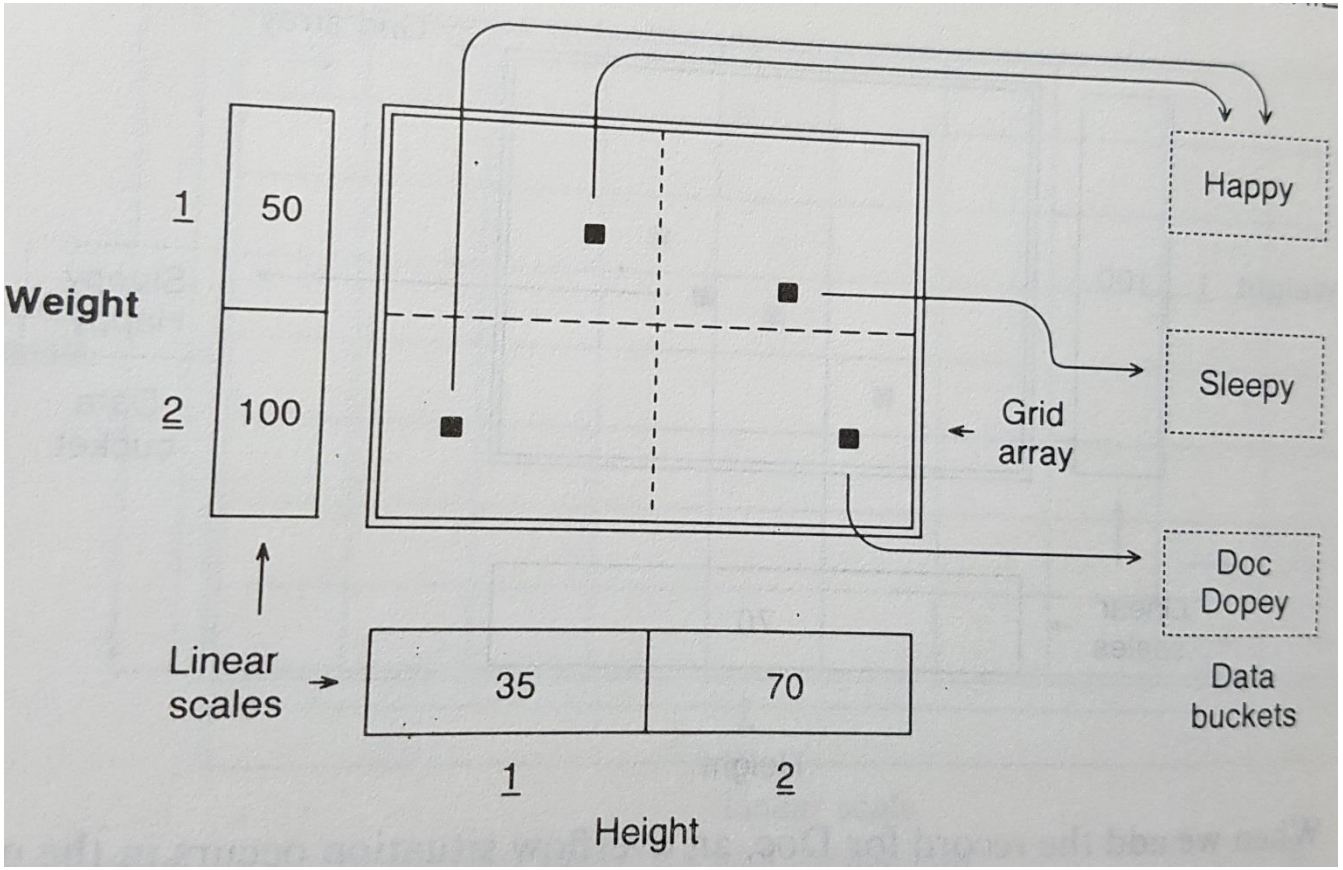
# Insertion (Cont'd)

- When we insert Doc(38, 51), an overflow occurs in the data bucket. We need to split it and partition the search space.
- We arbitrarily choose height dimension first to split, and we alternate dimensions for subsequent partitionings.
- We choose the midpoint of the height dimension for partitioning, then the linear scale is [35; 70].
- Happy maps to the 1st interval, Sleepy and Doc go to the 2nd.

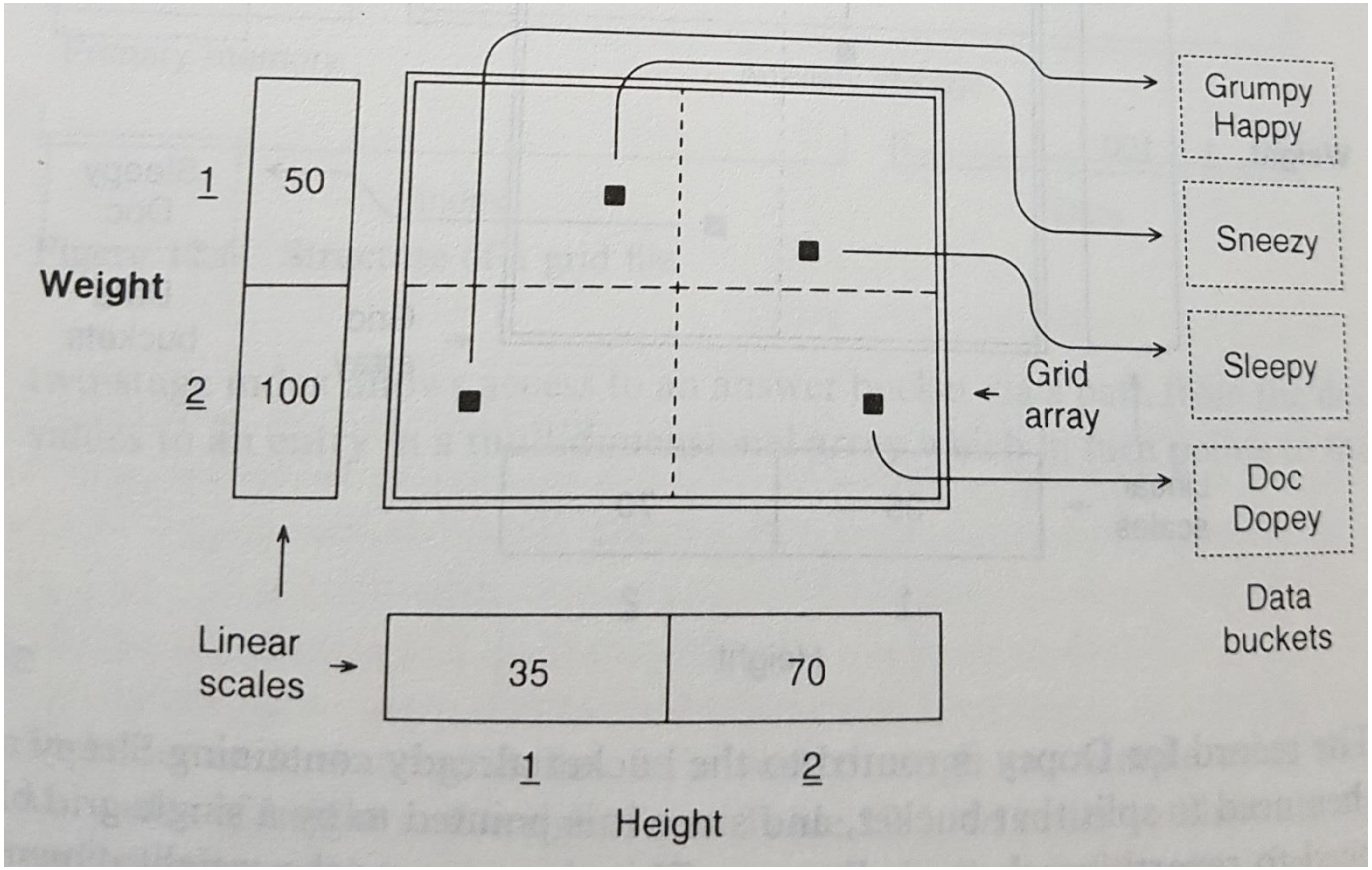
# Insertion (Cont'd)



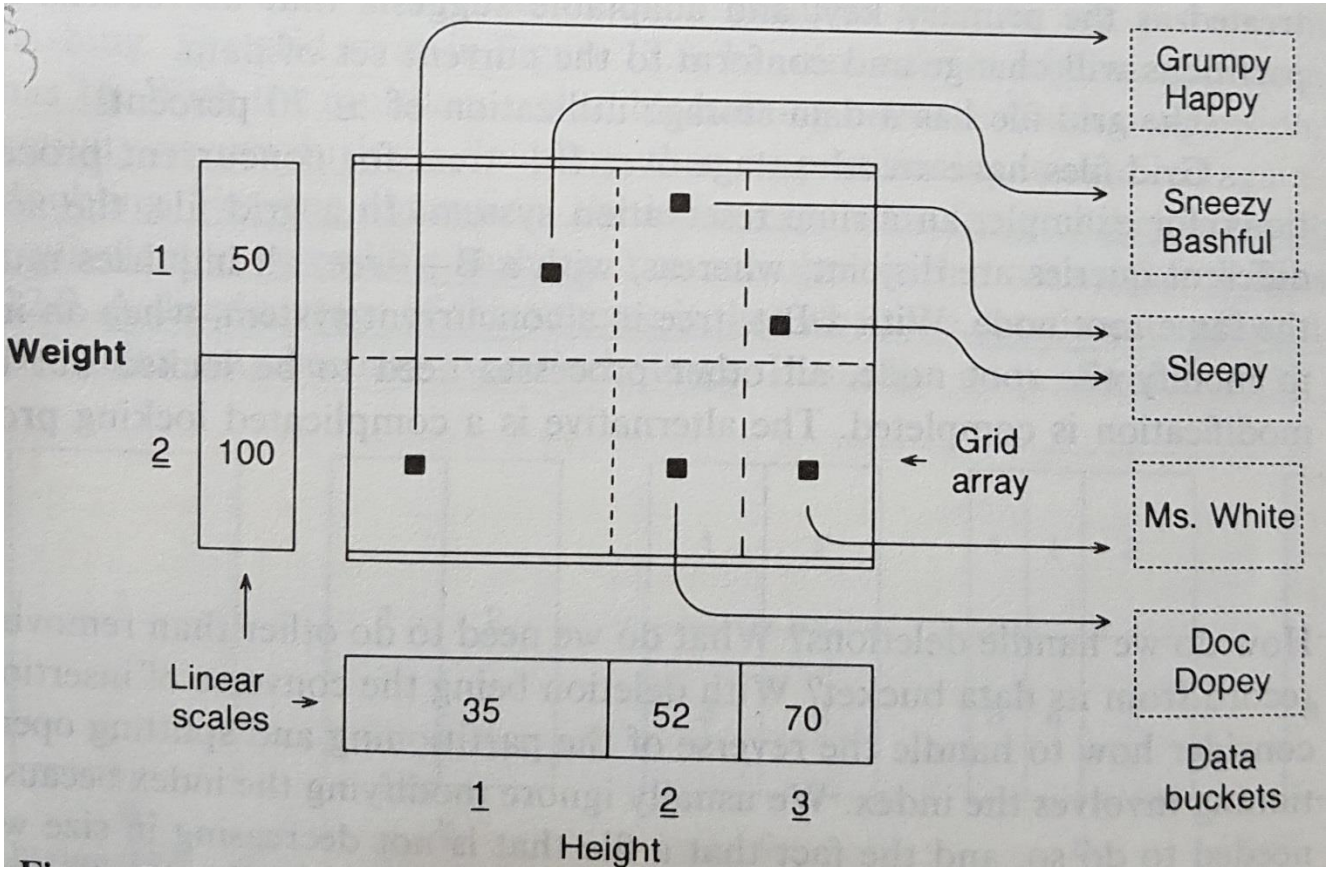
# Insert Dopey(37,54)



# Insert Grumpy, then Sneezy



# Insert Bashful, then Ms. White



# Grid files

- ⊕ Good for multiple-key search.
- ⊕ Suitable for concurrent processing applications, access paths for different queries are different.
- ⊕ Supports both range and equality search.
- ⊖ Space, management overhead (nothing is free)
- ⊖ Need partitioning ranges that evenly split keys.